SOFTWARE ENGINEERING

LAB 5: STATIC CODE ANALYSIS

Movie Reviews Program — Analysis, Fixes, and Reflection

Name: KAVAN S
SRN: PES2UG24CS223
Section: D
Date: 14/10/2026

1. Objective

The objective of this lab is to improve the quality, security, and style of a Python program using static code analysis tools. Pylint, Bandit, and Flake8 were used to analyze the Movie Reviews program, identify potential problems, apply fixes, and verify the updated source code.

2. Introduction

Static code analysis is the process of examining source code without executing the program. It helps identify potential programming errors, security vulnerabilities, and coding-style violations.

The following tools were used in this lab:

- **Pylint:** Checks code quality, naming conventions, documentation, and potential programming issues.
- **Bandit:** Identifies potentially insecure coding patterns in Python programs.
- **Flake8:** Checks Python coding style, whitespace, formatting, and common errors.

These tools help developers improve code readability, maintainability, and security.

3. Tools and Final Verification

Tool	Purpose	Final Result
Pylint	Code quality and potential programming issues	10.00/10
Bandit	Security analysis	No issues identified
Flake8	Coding style and formatting	No violations reported

The final Pylint report gave a score of 10.00/10. The Bandit report identified no issues, and the direct Flake8 run produced no output, indicating that no violations were reported.

4. Issues Identified and Fixes Applied

No.	Issue	Description	Fix Applied
1	Hardcoded bind address	The original program used 0.0.0.0, which configures a service to listen on all network interfaces.	Changed STREAM_HOST to 127.0.0.1 for localhost-only configuration.
2	Hardcoded password	The original source contained a password directly in the code.	Read MODERATOR_PASSWORD from an environment variable.
3	Weak hash algorithm	The original program used MD5 to generate review identifiers.	Replaced MD5 with SHA-256.
4	Standard pseudo-random selection	The original program used random.choice() to select a featured movie.	Replaced it with secrets.choice().
5	Function naming conventions	Several original function names used camelCase instead of Python's conventional snake_case.	Renamed functions to snake_case.
6	Missing documentation	The original module and functions lacked documentation strings.	Added a module docstring and function docstrings.
7	Global state modification	The original featured-movie function modified a global variable.	Removed the global variable and returned the selected movie directly.
8	Whitespace and comment spacing	The original source contained inconsistent spacing around operators, comments, and commas.	Corrected spacing to follow Python style conventions.
9	Blank lines between functions	Function definitions did not consistently follow the required blank-line spacing.	Adjusted blank-line spacing between functions.
10	Long lines	Some lines exceeded the configured line-length limit.	Split long conditions across multiple lines.

Conclusion

In this lab, the Movie Reviews program was examined using Pylint, Bandit, and Flake8. The identified issues were addressed in the updated source code, and the tools were run again for verification.

The final results were a Pylint score of 10.00/10, no issues identified by Bandit, and no violations reported by Flake8. This lab demonstrated that static analysis is useful for improving code quality, security, and maintainability. However, static analysis complements rather than replaces runtime testing and security review.

GitHub Repository

Repository Link: https://github.com/PES2UG24CS223/SE-LABS

Lab Folder: Lab5_Static_Code_Analysis